

1) LL(1) Parser Design

Grammar:

$S \rightarrow ABD$

$A \rightarrow a | Db | \epsilon$

$B \rightarrow g | dA | \epsilon$

$D \rightarrow e | f$

i) First & Follow steps

First:

* $\text{First}(S) = \{a, d, e, f, g, \epsilon\}$

* $\text{First}(A) = \{a, e, f, \epsilon\}$

* $\text{First}(B) = \{g, d, \epsilon\}$

* $\text{First}(D) = \{e, f\}$

Follow:

* $\text{Follow}(S) = \{\$ \}$

* $\text{Follow}(A) = \{d, e, f, \$ \}$

* $\text{Follow}(B) = \{e, f\}$

* $\text{Follow}(D) = \{b, e, f, \$ \}$

ii) Predictive Parsing Table

NT.	a	d	e	f	g	\$
S	ABD	ABD	ABD	ABD	ABD	ABD
A	a	ϵ	Db	Db	-	ϵ
B	-	dA	ϵ	ϵ	gD	-
D	-	-	e	f	-	-

iii) Verify LL(1)

- * First / First conflict = None
 - * First / Follow conflict = None
 - * Multiple table entries = None
- $\therefore$ This grammar is LL(1)

iv)

Predictive Parsing

Input: **adtf** \$

\$ $\rightarrow$ ABD \$

A $\rightarrow$ a (match a)

B $\rightarrow$ dA (match d)

A $\rightarrow$ ϵ

D $\rightarrow$ f (match f)

Input is accepted

v)

Design Recommendation

- * Grammars can be used directly for LL(1) parsing
- * No grammar modification is needed

Advantages:

- * Simple Implementation
- * No backtracking
- * Fast parsing
- * Easy error detection
- * Efficient syntax analysis

Annexure A

SHORT ANSWER TEST

Q1. A lexical analyzer produces the token stream:

$\text{id} + \text{id} * \text{id}$

Explain how the parser validates the syntactic structure of the expression.

Q2. Given the grammar:

$S \rightarrow aSb \mid \epsilon$

Determine whether the string $aaabbb$ belongs to the language using derivation.

Q3. Remove left recursion from the grammar:

$E \rightarrow E + T \mid T$

Q4. Find $\text{FIRST}(A)$ for the grammar:

$A \rightarrow aB \mid \epsilon$

$B \rightarrow b$

Q5. Determine whether the grammar is suitable for top-down parsing:

$A \rightarrow Aa \mid b$

Q6. Apply left factoring to the grammar:

$S \rightarrow \text{while } E \text{ do } S \mid \text{while } E \text{ do } S \text{ else } S$

Q7. Construct a parse tree for the expression:

$\text{id} * \text{id} + \text{id}$

using the grammar:

$E \rightarrow E + E \mid E * E \mid \text{id}$

Q8. Write the recursive descent procedures for the grammar:

$S \rightarrow aA$

$A \rightarrow bA \mid c$

Q9. Given the grammar:

$S \rightarrow AB$

$A \rightarrow a \mid \epsilon$

$B \rightarrow b$

Construct the predictive parsing table.

Q10. Demonstrate the first three actions of a shift-reduce parser for:

$\text{id} + \text{id}$

Q11. Identify the handle in the expression:

$\text{id} * \text{id} + \text{id}$

for the grammar:

$E \rightarrow E + E \mid E * E \mid \text{id}$

Q12. Construct the operator precedence relations for the operators:

$+$, $*$

where $*$ has higher precedence than $+$.

Q13. Explain how operator precedence parsing handles the expression:

2/4



$\text{id} + \text{id} * \text{id}$

Q14. What are LR parsers? Illustrate with a simple parsing scenario.

Q15. Given the grammar:

$S \rightarrow AA$

$A \rightarrow aA \mid b$

Write the steps involved in constructing an SLR parsing table.

Q16. Compute $\text{FOLLOW}(A)$ for the grammar:

$S \rightarrow AB$

$A \rightarrow a \mid \epsilon$

$B \rightarrow b$

Q17. Construct the augmented grammar for:

$S \rightarrow aA$

$A \rightarrow b$

Q18. Find the closure of the $\text{LR}(0)$ item:

$S' \rightarrow \cdot S$

for the grammar:

$S \rightarrow aA$

$A \rightarrow b$

Q19. Check whether the grammar is ambiguous:

$E \rightarrow E + E \mid \text{id}$

using the string:

$\text{id} + \text{id} + \text{id}$

Q20. Construct a leftmost derivation for:

$\text{id} + \text{id} * \text{id}$

using

$E \rightarrow E + E \mid E * E \mid \text{id}$

Q21. Apply left factoring to:

$S \rightarrow aBc \mid aBd \mid b$

Q22. Find $\text{FIRST}(F)$ for:

$F \rightarrow (E) \mid \text{id}$

Q23. Determine whether the grammar is suitable for predictive parsing:

$S \rightarrow aA \mid aB$

$A \rightarrow b$

$B \rightarrow c$

Justify your answer.

Q24. Construct recursive descent parser procedures for:

$S \rightarrow abS \mid \epsilon$

Q25. Build the predictive parsing table for:

$S \rightarrow aS \mid b$

and show the table entries for terminals a , b , and $\$$.

2) Shift-Reduce Parser Design

$E \rightarrow E + E \mid E - E$

$E \rightarrow E * E \mid E / E$

$E \rightarrow (E)$

$E \rightarrow id$

(i) Grammar: This grammar supports arithmetic expressions (+, *, -, /, (,), id)

(ii) Shift Reducing Parsing:

Stack	Input	Action
\$	id + id * id \$	shift id
\$ id	+ id * id \$	Reduce $E \rightarrow id$
\$ E	+ id * id \$	shift $E \rightarrow id$
\$ E +	* id * id \$	shift id
\$ E + id	* id \$	Reduce $E \rightarrow id$
\$ E + E	* id \$	shift *
\$ E + E +	id \$	shift id
\$ E + E * id	\$	Reduce $E \rightarrow id$
\$ E + E + E	\$	Reduce $E \rightarrow E * E$
\$ E + E	\$	Reduce $E \rightarrow E + E$
\$ E	\$	Accept

(iii) Shift and Reduce Operations

* Shift: Push the next input symbol onto stack

* Reduce: Replace symbols on the stack using a grammar production

(iv) Parser Result:

The input $id + id * id$ is Accepted

(v) Advantages of Shift Reduce Parser

- * Handles operator precedence
- * Suitable for bottom up parsing
- * Detects syntax errors efficiently
- * Used in LR, SLR and LALR parsers

3) Recursive Descent Parser.

$S \rightarrow (L) la$

$L \rightarrow S L'$

$L' \rightarrow , S L' | \epsilon$

Input string: (a)

i) Recursive Procedures

$S() :$

if '(' match ('('), $L()$, match (')')

else if 'a' match ('a')

$L() :$

$S()$

$L'()$

$L'() :$

if ',' match (','), $S()$, $L'()$

else ϵ

Top-Down Parsing:

Start with s

$s \Rightarrow (L)$
 $\Rightarrow (SL')$
 $\Rightarrow (aL')$
 $\Rightarrow (a)$

(iii)

Function Calls and Terminal Matching

Steps	Action
1	Call $s()$
2	Match $($
3	Call $L()$
4	Call $S()$
5	Match a
6	$L' \rightarrow \epsilon$
7	Match $)$
8	Accept

(iv)

Result:

The input string (a) is Accepted because it is derived using $s \rightarrow (L) \rightarrow (SL') \rightarrow (aL') \rightarrow (a)$

or

Error Detection and Recovery

- * If the expected terminal does not match the input, display "Syntax Error"
- * Skip the invalid symbol or stop parsing and continue from the next valid symbol
- * Reject the input if parsing cannot continue

4) Operator Precedence Parser

Input: $id - id * id$

(i) Unambiguous Grammar

$E \rightarrow E - T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

(ii) Precedence Relations

Symbols	Relation
$id > -$	Reduce
$id > *$	Reduce
$- < id$	Shift
$- < ($	Shift
$* < id$	Shift
$* < ($	Shift
$* > -$	Reduce
$(< id$	Shift
$(=)$	Match
$) > -$	Reduce
$) > *$	Reduce

(iii) Operator Precedence Table

	id	$-$	$*$	$($	$)$	$\$$
id	$-$	$>$	$>$	$-$	$>$	$>$
$-$	$<$	$>$	$<$	$<$	$>$	$>$
$*$	$<$	$>$	$>$	$<$	$>$	$>$
$($	$<$	$<$	$<$	$<$	$=$	$-$
$)$	$-$	$>$	$>$	$-$	$>$	$>$
$\$$	$<$	$<$	$<$	$<$	$-$	Accept

Stack Based Parsing Algorithm

- 1) Push \$ onto the stack
- 2) Compare the precedence of the stack top with next symbol
- 3) if $<$ or $=$, shift input symbol
- 4) if $>$, Reduce using grammar
- 5) Repeat until the stack contains \$E and input is \$

Step by Step Shift-Reduce Parsing

Stack	Input	Action
\$	id - id * id \$	Reduce $F \rightarrow id$ shift id $T \rightarrow E$
\$id	- id * id \$	Reduce $F \rightarrow id \rightarrow T \rightarrow E$
\$E	- id * id \$	Shift -
\$E-	id * id \$	Shift id
\$E-id	* id \$	Reduce $F \rightarrow id \rightarrow T$
\$E-T	* id \$	Shift *
\$E-T*	id \$	Shift id
\$E-T*id	\$	Reduce $F \rightarrow id$
\$E-T*F	\$	Reduce $T \rightarrow T * F$
\$E-T	\$	Reduce $E \rightarrow E-T$
\$E	\$	Accept

7) Design and Construct an LALR(1) Parser

$$S \rightarrow AA$$

$$A \rightarrow aA \mid b$$

8) Augmented Grammar

$$S' \rightarrow S$$

$$S \rightarrow AA$$

$$A \rightarrow aA$$

$$A \rightarrow b$$

(ii) Canonical LR(1) items

* $I_0: S' \rightarrow \cdot S, S \rightarrow \cdot AA, A \rightarrow \cdot aA, A \rightarrow \cdot b$

$I_1: S' \rightarrow S \cdot$

$I_2: S \rightarrow A \cdot A, A \rightarrow \cdot aA, A \rightarrow \cdot b$

$I_3: A \rightarrow a \cdot A, A \rightarrow \cdot aA, A \rightarrow \cdot b$

$I_4: A \rightarrow b \cdot$

$I_5: S \rightarrow AA \cdot$

$I_6: A \rightarrow aA \cdot$

States with identical cores are merged to obtain LALR(1) parser

FIRST and FOLLOW Sets

FIRST : $\text{FIRST}(S) = \{a, b\}$

$\text{FIRST}(A) = \{a, b\}$

FOLLOW :

$\text{FOLLOW}(S) = \{\$ \}$

$\text{FOLLOW}(A) = \{a, b, \$ \}$

LALR(1) Parser Table

* ACTION : Shift on a, b; Reduce $A \rightarrow b, A \rightarrow aA, S \rightarrow AA$;

Accept on \$

* GOTO : $S \rightarrow I_1, A \rightarrow I_2$

Passing for Input aabb

\$	aabb\$	shift a
\$a	abb\$	shift a
\$aa	bb\$	shift b
\$aaA	b\$	Reduce $A \rightarrow b$
\$aA	b\$	Reduce $A \rightarrow aA$
\$A	b\$	shift b
\$AA	\$	Reduce $A \rightarrow b$
\$S	\$	Reduce $S \rightarrow AA$
\$S	\$	Accept

Production Used:

1) $A \rightarrow b$

2) $A \rightarrow aA$

3) $A \rightarrow b$

4) $S \rightarrow AA$

Result:

Input string aabb is accepted because it is reduced to start symbol S